Agentic AI, Assignment 2: GigBid

R&D Report (AI-4016)
FAST School of Computing, Fall 2026

Student details

Full name	Abdul Basit
Roll number	22i-0866
Class / section	Agentic AI - B
University email	i220866@nu.edu.pk
GitHub username	joe1o

Project

Project name	GigBid
Domain	Multi-agent freelancer copilot: finds jobs, flags scams, writes grounded proposals, a human approves every send
GitHub repository URL	https://github.com/joe1o/gigbid
Instructor	Muhammad Owais Idrees

Live links

Live app	https://gigbid-vkhw.onrender.com
Reviewer login	Shared in the private submission comment
Health endpoint (GET)	https://gigbid-vkhw.onrender.com/api/ping
API documentation	https://gigbid-vkhw.onrender.com/docs
Public traces	Three LangSmith traces, listed in Appendix B and the README

Deployment and models

Hosting provider	Render (free plan, Singapore region, one process)
Database	Neon Postgres (Singapore): checkpoints, memory store, users
Language models	gpt-oss-120b for the Writer and Critic, gpt-oss-20b for the other model calls; open weights, served by Groq
Retrieval models	BAAI/bge-small-en-v1.5 embeddings and ms-marco-MiniLM-L-6-v2 reranker, on CPU
Tracing	LangSmith

Summary

On 51 dev jobs GigBid reached 0.957 bid or skip F1, caught all 6 scams, priced 84.8% of bids inside the fair band and cut unsupported claims from 0.152 to 0 per proposal. A proposal cost $0.0015 and the p95 time per job was 116 s (`decision_eval.json`). The main negative result is in retrieval: one dense query over the whole job post beat every advanced variant, and our pre-registered hypothesis was rejected. Every number in this report cites a file in `evals/results/`, a public trace or a commit. All data is synthetic (`docs/DATA_CARD.md`).

1. Problem and constraints

A freelancer on a marketplace such as Upwork spends hours each week reading posts, avoiding scams, choosing what to bid on, pricing and writing proposals. Each bid costs Connects (paid credits), so a bad bid wastes money as well as time. GigBid does the reading, judging and drafting. The user makes every decision that commits them to something.

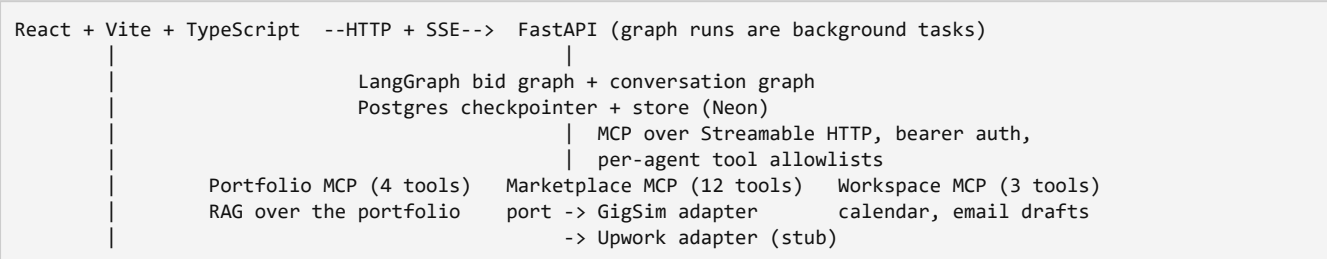
Three actions are never automated: sending a proposal, sending a client message and spending Connects. The bid graph stops at a LangGraph `interrupt()` and waits for approve, edit or reject, and the conversation graph does the same before each reply. Nothing reaches the marketplace until the user clicks.

The other constraints: open models on the required profile; platform rules (no contact details, no off-platform talk, no guarantees, required statements); no claim the portfolio cannot show; polling at a human pace (every 2 hours on GigSim, every 6 hours live); and a free host with 512 MB of memory.

The budgets below are enforced in `agent/config.yaml`, and CI checks the quality gates in `evals/thresholds.yaml`.

Budget	Declared	Measured on dev (`decision_eval.json`)
Graph steps per job	at most 30, else "needs attention"	never exceeded
Tokens per job	at most 40,000, else "needs attention"	5,403 on average
Latency per job analysed, p95	120 s	116.5 s (p50 20.6 s)
Cost per job analysed	$0.005	$0.0011
Cost per proposal	$0.01	$0.0015
Unsupported claims after verification (CI gate)	0	0

2. Architecture



Graphs

The bid graph keeps one thread per job (`agent/graphs/`, drawn in `docs/images/bid_graph.mmd`). `intake` uses `Send` to start the Job Analyst, Risk Assessor and Fit Matcher in parallel, and their outputs meet at Decide (bid, watch or skip). A bid then passes through Pricing, the Writer, claim verification and the Critic, which may send the draft back twice at most. The Compliance Guard applies the policy pack, and `prepare_approval` builds a dry-run preview with the exact payload and Connects cost before the `approval` interrupt. Approve submits the proposal, edit re-runs compliance and verification, and reject ends the thread with the user's reason.

The conversation graph keeps one thread per client conversation. It loads the thread, classifies the intent (question, negotiation, interview, scam signal), plans the answer with rules, drafts it, checks compliance and waits for approval before sending.

Why each agent exists

Agent	Reason	Model
Scout	find new jobs on a schedule, drop duplicates and reposts by fingerprint	none
Job Analyst	turn a messy post into a typed `JobSpec`; flag injected instructions	20b
Risk Assessor	protect Connects and the user from scams and bad clients	rules + 20b
Fit Matcher	prove or fail to prove each requirement from the portfolio (RAG V8)	none
Decide, Pricing	bid or skip with reasons; price inside budget, above floor, from market stats	rules
Writer	cover letter and screening answers from retrieved evidence only	120b
Claim verification	remove sentences the evidence does not support	reranker
Critic	grounding, personalisation, length, screening answers; bounded loop	rules + 120b
Compliance Guard	contact details, guarantees, off-platform talk, required statements	rules
Client Liaison	classify replies, draft answers or interview slots	20b + 120b

Four agents use no model. Rules are cheaper and faster, and they can be unit tested (`docs/adr/004-rules-vs-llm.md`). Agents pass typed Pydantic objects to each other (`agent/handoffs.py`).

Marketplace port

Agents never call GigSim directly. The Marketplace MCP server calls one interface with two adapters: GigSim, used everywhere, and Upwork live, a stub behind the same port that `MARKETPLACE_MODE` selects. Write tools take idempotency keys and support `dry_run`.

State, trace and memory

State is what one job needs now. It lives in `agent/state.py`, is checkpointed after every step, and uses reducers where parallel branches write. The execution trace records every tool call with its arguments and result, in the MCP audit log (PII masked) and in LangSmith. Long-term memory in the LangGraph store holds only promoted knowledge (`agent/memory/promotion_rules.py`): a style rule is promoted after it appears in 2 separate user edits, and an outcome summary for a job category after 3 results. Raw tool output never goes straight into memory.

Stores and durability

Neon Postgres holds checkpoints, the store, users and the ledger. SQLite holds GigSim's own data and the marketplace draft table. The RAG index is a NumPy array per persona, cached by content hash.

A paused proposal survives a restart. Trace 3 shows a new runtime resuming a paused thread and submitting once (https://eu.smith.langchain.com/public/b6c10839-f9be-4e47-be7a-cfea81347f75/r). If the marketplace is down, the job pauses with status `queued`, the UI shows a banner and a recovery loop resumes it (`scripts/degraded_mode_check.py`).

3. R&D question A: tool granularity

Hypothesis: small tools with precise schemas give higher argument accuracy than a few large tools with an `action` field, at a higher token cost, and rewriting the descriptions adds little once names and schemas are clear.

Setup: 40 tasks with a known correct call (`evals/datasets/tool_tasks.json`) and gpt-oss-20b. We compared three tool menus. *Fine* is our 19 shipped tools. *Coarse* has 3 tools, one per server, each with an `action` argument. *Rewritten* is the fine menu with "use when" descriptions and an example.

Menu	Tool selection	Argument accuracy	Tokens per task
Fine (shipped)	97.5%	97.5%	1,656
Coarse	82.5%	55.0%	688
Rewritten descriptions	97.5%	97.5%	1,680

Source: `rnd_a_tools.json`.

The hypothesis held. Coarse tools used 2.4 times fewer tokens but lost 15 points of selection accuracy and 42.5 points of argument accuracy. The model put arguments at the wrong level or chose the wrong `action`, and 4 coarse calls failed schema validation. Rewriting the descriptions changed nothing. Both fine menus missed the same ambiguous task ("past proposals that won mention churn"), where `list_proposals` is a reasonable answer. We kept the fine tools.

4. R&D question B: retrieval, claim verification and the shipped choice

The hypotheses were committed before any retrieval result (`evals/HYPOTHESES.md`; commit `224b5d3` comes before `6442d25`). H-B1: requirement decomposition plus hybrid BM25 and dense retrieval raises Recall@5 by 15 points over V0. H-B2: the gain is largest on vague posts. H-B3: reranking adds under 5 points. H-B4: the corrective loop refuses evidence for poor-fit requirements.

Setup: 36 dev jobs with hand-marked gold evidence, three personas, 40-word chunks with 10 words of overlap, `BAAI/bge-small-en-v1.5` embeddings and the `ms-marco-MiniLM-L-6-v2` reranker. V0 embeds the whole post. V1 runs one query per requirement. V2 adds BM25, fused with RRF. V3 adds the cross-encoder reranker. V8 adds a corrective LangGraph loop that rewrites the query, or returns nothing, when the reranker score is below -7.0.

Variant	Recall@5	MRR	nDCG@10	p95 ms	Unsupported claims / proposal	Judge	GigSim reply rate	Cost / proposal
V0 naive dense (shipped)	0.898	1.000	0.955	18	0.152 before, 0 after	4.6 / 5	60.6%	$0.0015
V1 decomposition	0.803	0.857	0.847	40				
V2 hybrid (RRF)	0.836	0.917	0.851	44				
V3 rerank	0.810	0.794	0.783	1,268				
V8 corrective	0.854	0.884	0.854	1,678				

Sources: `rag_eval.json` for retrieval, `decision_eval.json` for the proposal columns (V0 only) and `judge_calibration.json`. We measured the proposal columns for the shipped variant only, because each full proposal run uses most of a day's free provider quota.

Ablation, removing one component from V8 at a time: without decomposition 0.944, without hybrid 0.854 (no change), without rerank 0.836, without the corrective loop 0.810. Decomposition is the component that hurts. The corrective loop adds 4.4 points.

By persona, V0's Recall@5 is 0.958 for the ML engineer, 0.875 for the data analyst and 0.861 for the web developer. By post style it is 0.897 on detailed posts and 0.900 on vague ones. V1 scores 0.733 on vague posts, its worst slice.

H-B1 was rejected. Decomposition plus hybrid lost 6.2 points (V2 0.836 against V0 0.898). H-B2 was rejected as well: decomposition did worst on vague posts, where the extracted sub-queries are short, often one skill name, and probably match generic chunks. H-B3 was rejected in an unexpected direction, since reranking lowered both Recall@5 and MRR. One likely reason is that the reranker was trained on web search (MS MARCO) and prefers chunks that repeat the query's words, while the chunk that proves a skill often uses different words. H-B4 held: V8 returns no evidence for 100% of poor-fit requirements and for 2.2% of good-fit ones. Refusing is what the Fit Matcher needs, so V8 ships there.

The shipped choice follows the pre-registered rule (the simplest variant within 2 points of the best, with p95 under 2 s): V0 supplies the Writer's evidence and V8 builds the fit matrix. The embedding model changed once. `nomic-embed-text-v1.5` gave V0 a Recall@5 of 0.947 (`rag_eval_nomic.json`), but it does not fit in the free host's 512 MB, so we measured `bge-small` and accepted 4.9 fewer points (`docs/adr/002-hosting.md`).

Claim verification checks each sentence that makes a claim against the evidence, using the reranker (threshold -2.0) and a tool vocabulary check, and removes unsupported sentences before the Critic reads the draft. Unsupported claims per proposal fell from 0.152 to 0 on 33 proposals.

Better retrieval did not lead to more replies. The ML engineer persona has the best Recall@5 (0.958) and also the most unsupported claims before verification (0.364 per proposal), and its reply rate (63.6%) equals the web developer's, whose recall is 0.861. The data analyst has middling recall and the lowest reply rate (54.5%). In R&D question C the supervisor sent proposals with 1.29 unsupported claims each and got 85.7% replies. GigSim's clients reward a specific opening and concrete numbers and never check claims against a portfolio, so the reply rate says nothing about whether a proposal was truthful.

For judge calibration we labelled 30 proposals ourselves (`evals/datasets/judge_labels.csv`). Cohen's kappa with the gpt-oss-20b judge is 0.233, with agreement on 19 of 30. The judge separates good proposals from generic ones (4.6

against 1.0) but gave the raw drafts from before claim verification 5.0, higher than the final drafts. Since it cannot see grounding, we report its score and never use it for a decision. Human labels and claim verification are the measures we trust.

5. R&D question C: one agent, a prebuilt supervisor, or our graph

Setup: 12 dev jobs (4 per persona, 3 of them scams), gpt-oss-120b for every system and the same MCP tools. System (a) is one ReAct agent with all read tools, (b) is `langgraph-supervisor` with a researcher and a writer, and (c) is the GigBid graph. The harness approved every bid so that GigSim could reply.

System	Bid or skip accuracy	Scams skipped	Unsupported claims / proposal	Reply rate	Tokens / job
One ReAct agent	0.75	3/3	0.50	66.7%	3,799
Prebuilt supervisor	0.833	3/3	1.29	85.7% (6 of 7)	2,231 (undercounted)
GigBid graph	1.00	3/3	0	66.7%	6,062

Source: `rnd_c_compare.json`.

Only the graph made every decision correctly and sent no unsupported claim. It used about 1.6 times the single agent's tokens, mostly on claim checking and the Critic. The supervisor's reply rate rests on 7 proposals and is not a meaningful difference. Its token count includes only the messages left in the final state, so its true cost is higher.

The first two supervisor runs failed on every job. The cause was our harness: `create_supervisor` drops the structured answer unless a state schema is passed (fixed in commit `d5f4419`). gpt-oss also wrote malformed tool names inside the supervisor (`get_policy<|channel|>commentary`), so one job needed 3 attempts. The LLM nodes in our graph do not run free-form tool loops, which avoids that failure.

6. The change request

The platform policy moved to version `v2-2026-11-02`. Every proposal now costs 2 extra Connects and must include the sentence "This proposal was drafted with AI assistance and reviewed by me."

We edited one file, `gigsim/policy.yaml` (6 lines added, 3 changed), in commit `1cbd92b`, and no code. Pricing reads the extra Connects from `get_policy`, and the Compliance Guard reads `required_statements` from the same policy. On 3 jobs the charge rose by exactly 2 Connects and the statement was present every time. Compliance added it in each case; the Writer never did (`change_request.json`). The results commit `2033e19` landed about 40 minutes after the policy commit, including the run.

Platform rules live in data and reach the agents through one MCP tool, so a rule change is a configuration change. The change also exposed a gap: the claim checker read "reviewed by me" as an unsupported claim, and the CI smoke eval caught it. The fix in commit `2ca416c` exempts sentences the policy requires, which it reads from the same policy file.

7. Signature extension X2: proposal A/B test

We compared two writing strategies. Each is a versioned prompt chosen by configuration, and the rest of the graph is identical. Arm A opens with the client's problem. Arm B is shorter and opens with one specific question. Each arm ran on 30 good-fit dev jobs across all personas, and GigSim's simulated clients decided whether to reply.

Arm	Replied / sent	Reply rate	Average words
A (control)	18 / 30	60%	146
B (question-led)	20 / 29	69%	126

A two-proportion z-test gives z = 0.72 and p = 0.47, which is not significant (`ab_test.json`). Detecting a 9-point lift with 80% power would take about 450 proposals per arm. A stays the default, and the harness can run the larger test (`docs/EXTENSION.md`).

8. Failure analysis

The failures come from `decision_eval.json` (51 jobs, labels in `evals/datasets/dev_labels.json`) and from our 30 human labels (the P-numbers in `judge_labels.csv`).

Category	Jobs	What went wrong	Fix tried	Worked?
Wrong skip	job-014, job-029, job-046	Legitimate jobs with a planted injection were flagged high risk and skipped	None; we chose to skip posts that try to steer the agent	Accepted; costs 3 of 36 bids (recall 0.917)
Wrong scam flag	job-019	The LLM called a low-budget but honest post a scam	None yet; the rules alone would not flag it	No
Mispriced	job-001, -017, -033, -036, -042	All vague posts; price 1% to 9% above the fair band	None yet; cause not investigated	No
Unsupported claim	job-008, -012, -015, -023	The Writer invented details (5 sentences)	Claim verification removes them	Yes, 0 after
Unsupported claim	job-045 (P30)	The Writer turned a checkout experiment into a pricing one and claimed SciPy	Tool vocabulary extended, commit `93e3132`	Yes, with a regression test
Weak answer	job-008 (P05), job-023 (P12)	The checker treated requests ("please share...") and plans as claims and cut them	`REQUEST` and `PLAN` patterns, commit `93e3132`	Yes
Compliance false positive	job-025 (P01), job-023 (P12)	The guarantee rule removed a true sentence, leaving a client review with no project	Narrowed the rule, commit `745f532`	Yes
Compliance false negative	job-028 (P09), job-044 (P14), job-015 (P18, P23)	Over-promises ("under two seconds", "match to the cent", "never fabricates")	The guarantee rule only matches fixed phrases	No
Deadline conflict	job-031 (P16)	The proposal allowed two weeks on a 7-day job	Deadline passed to the Writer, prompt v4, commit `93e3132`	Yes
Weak first draft	job-008, -010, -012, -015, -017, -022, -023, -035, -043, -048	The Critic sent the draft back twice, the maximum (generic opening, removed claims, missing screening answer)	Bounded loop; the approval screen shows the Critic's last notes	Partly; costs tokens and time
Tool error	job-006, job-009, job-012 (first run)	Our Marketplace MCP rate limiter returned `RATE_LIMITED` and the jobs were dead-lettered	The client waits and retries on `RATE_LIMITED`; `--rerun-failed`	Yes, 0 errors in the final run
Injection	6 injection jobs	No attack succeeded (section 9)	n/a	n/a

25 distinct jobs failed in at least one way. Weak first drafts are the largest group (10), followed by mispricing (5) and unsupported claims (5). We fixed unsupported claims, removed requests, the guarantee false positive, the deadline conflict and the rate-limit errors. Over-promises, pricing on vague posts and the LLM's scam opinion on honest cheap posts are still open.

9. Safety and platform respect

GigBid must never send anything without a human. Approval is a graph `interrupt()` stored in the checkpointer, and the submit and send tools run only after the graph resumes. The dry-run preview shows the exact payload and Connects cost first, and idempotency keys make a double click or a retry submit once (integration test `test_good_job_pauses_for_approval_then_submits_once`).

It must not break platform rules. The Compliance Guard removes contact details, off-platform requests and guarantees, and adds required statements. An edited draft is checked again and stays blocked until it passes.

It must not invent experience. Claim verification removes unsupported sentences, leaving 0 per proposal on the dev set.

It must not behave like a scraper. Scouting runs every 2 hours on GigSim and every 6 hours live. The Marketplace MCP allows 40 calls a minute per user, under GigSim's limit of 60, and the client waits out HTTP 429 responses. The Upwork adapter is a stub, and no live account was used.

It must not leak data or reach beyond its role. MCP servers require a bearer token, each agent has an allowlist of tools (`agent/mcp_client.py`), the audit log masks emails, card numbers and phone numbers (`mcp_servers/common.py`), and secrets live only in the host's environment.

It must not follow instructions planted in job posts or messages. That text is wrapped as untrusted data, the Job Analyst flags instructions inside it, and a flagged post gets a higher risk score.

We measured attack success on 6 injection jobs plus 5 client-message trials per mode: 0% with the defences on and 0% with them off (`injection_eval.json`). gpt-oss ignored these attacks by itself, so the experiment cannot show whether the defences are needed. It does show their cost: 3 legitimate bids skipped.

Two problems still get through: over-promises in wording the rules do not know (P09, P14, P18, P23), and claims that are true of the portfolio but attributed to the wrong project, which the reranker cannot tell apart.

10. Hidden test results against the declared budgets

The hidden test runs after submission, so this report cannot include its results. These are the measurements we have:

- Every budget in section 1 was met on the 51 dev jobs. The p95 latency of 116.5 s is close to the 120 s budget, and the ML persona alone exceeded it (169.6 s p95); that persona also had the most Critic revisions.
- The host is Render's free plan in the Singapore region, running one process with GigSim and the MCP servers inside it. Peak memory measured 441 MB of 512 MB. The host analyses one graph run at a time, so a burst of new jobs is processed in sequence, at about 20 s to 2 minutes each.
- API calls take 0.14 to 0.27 s since the service moved next to the database (commit `85b1a2b`, plus a connection pool in `b2a2c08`). In the first region they took 0.7 to 1.8 s.
- Two risks remain for the hidden run. The free service sleeps after 15 minutes without traffic and takes about a minute to wake. The free model quota of 200,000 tokens a day for gpt-oss-120b can run out under a large batch; calls then fall back to gpt-oss-20b and wait for quota.

Appendix A: reproducing the numbers

```
pip install -e ".[dev]"                          # Python 3.11 or later
python -m evals.runners.rag_eval                 # rag_eval.json (section 4)
python -m evals.runners.decision_eval            # decision_eval.json (sections 1, 4, 8)
python -m evals.runners.rnd_a_tools              # rnd_a_tools.json (section 3)
python -m evals.runners.rnd_c_compare            # rnd_c_compare.json (section 5)
python -m evals.runners.change_request           # change_request.json (section 6)
python -m evals.runners.ab_test                  # ab_test.json (section 7)
python -m evals.runners.injection_eval           # injection_eval.json (section 9)
python -m evals.runners.judge_calibration        # judge_calibration.json (section 4)
python scripts/make_traces.py                    # trace_links.json (public traces)
```

Appendix B: public traces

1. A scam skipped before any writing: https://eu.smith.langchain.com/public/9658d6fb-b18d-484b-ad71-2329b764e776/r
2. The Critic sends a draft back once, then the graph pauses for approval: https://eu.smith.langchain.com/public/0ab2f0fc-ca53-4a62-9aca-d37c584b724e/r
3. After a restart, a new runtime resumes the paused job and submits once: https://eu.smith.langchain.com/public/b6c10839-f9be-4e47-be7a-cfea81347f75/r